HashBench: Empirical Performance Evaluation of Separate Chaining, Linear Probing, and Robin Hood Hashing Under Dynamic Key Workloads

Huỳnh Anh Tuấn*(TV1, QE200073)* • **Hồ Anh Nhật***(TV2, QE200301)* • **Đinh Thiện Nhân***(TV3, QE190092)* • **Nguyễn Trần Anh Kiệt***(TV4, QE200105)* • **Châu Minh Tiến***(TV5, QE200077)* • **Võ Ngọc Phú***(TV6, QE200004)*

FPT University – Fall 2026 Semester – Class SE20C – Group 2 (Variant B)

Date: October 29, 2026

Abstract*—We evaluate Separate Chaining (SC), Linear Probing with tombstones and optional rebuild (LP-Tomb), Linear Probing with backward-shift deletion (LP-Backshift), and Robin Hood hashing (with and without early-stop) using Java 8 and 12-character String keys. The benchmark comprises 560 isolated JVM processes across three core research questions. At load factor α = 0.50, measured probe counts for Linear Probing are 1.5009 for successful lookups and 2.5026 for unsuccessful lookups, matching Knuth's theoretical estimates within 0.1% error. In RQ1, SC exhibits lower median lookup time than LP for unsuccessful queries across all load factors; for successful lookups, LP is faster at α = 0.50 and 0.75 due to hardware cache locality, whereas SC dominates at α = 0.90 and 0.95 where LP suffers catastrophic primary clustering. In RQ2, tombstones reach 50.0% of the table after 4M delete-insert cycles, leading to empty slot exhaustion and 12 process timeouts. In RQ3 at live load 0.8 and C = 4M, median workload times are 9.413 s for LP-Tomb with rebuild (τ = 0.1), 5.901 s for LP-Backshift, 6.056 s for Robin Hood, and 6.025 s for Robin Hood without early-stop. LP-Tomb incurs 23 rebuilds with maximum block latency reaching 121.106 ms, while backward-shift and Robin Hood maintain smooth block latencies under 12.7 ms. We report all raw data, sealed under tag measure-v1, along with one timing-variance outlier verified via 11 deterministic rerun samples.

***Keywords*—**Hash Table, Linear Probing, Separate Chaining, Tombstone Lifecycle, Rebuild Dynamics, Robin Hood Hashing, Backward-shift Deletion, Cache Locality, Pointer Chasing, Java 8.

I. INTRODUCTION

Hash tables serve as indispensable core data structures across systems software, database engines, and distributed computing frameworks, delivering expected O(1) amortized time complexities for insertions, deletions, and retrievals. Nevertheless, under dynamic, high-churn operating regimes characterized by prolonged sequences of interleaved deletions and insertions, classical collision resolution strategies reveal profound divergences in algorithmic probe counts, hardware cache utilization, and tail latency stability.

The two foundational collision resolution paradigms in computer science are *Separate Chaining* (SC), which resolves collisions outside the primary array via linked bucket chains, and *Linear Probing* (LP), which resolves collisions by sequentially scanning contiguous cells within a flat array. In open addressing, removing a key requires marking the vacated cell with a tombstone marker (TOMB) to preserve search path integrity for previously inserted colliding elements. Over millions of delete-insert operations, tombstone accumulation gradually exhausts available empty cells (EMPTY), degrading unsuccessful lookup operations into degenerate linear scans of length O(M) and forcing costly whole-table rebuilds. To eliminate tombstones while mitigating probe sequence variances, *Robin Hood hashing* (RH) and *backward-shift deletion* (Knuth's Algorithm R) have emerged as premier alternatives.

This paper reports the empirical findings from the HashBench experimental campaign, consisting of 560 isolated JVM benchmark executions conducted on standardized hardware (Intel Core i7-10850H, 31.8 GB RAM, Windows 11 Pro, Eclipse Adoptium Temurin JDK 1.8.0_504) specifically examining **Variant B** (12-character alphanumeric String keys). All experimental raw data have been formally sealed under Git tag `measure-v1` in `results/raw/official/`.

II. BACKGROUND & RELATED WORK

A. Classical Hash Table Algorithms

Separate Chaining (SC): The table maintains M = 2^p buckets, where each bucket anchors a singly-linked list of entry nodes. Upon a collision, the new key-value pair is inserted at the head of the chain in O(1) time following a traversal to confirm key uniqueness. Under the Simple Uniform Hashing Assumption (SUHA), Knuth [1] established that the expected probe lengths for successful lookups (S_SC) and unsuccessful lookups (U_SC) satisfy:

$$S_SC \approx 1 + \alpha / 2, \qquad U_SC \approx \alpha \tag{1}$$

where α = N / M denotes the load factor. The distinctive strength of SC is its linear degradation with respect to α and its complete immunity to clustering phenomena.

Linear Probing (LP): All keys reside directly within a flat array of size M. The initial bucket index is computed via 64-bit Fibonacci golden-ratio multiplicative hashing:

$$h(k) = \lfloor ((hashCode(k) \times C_\phi) \bmod 2^{64}) / 2^{(64 - p)} \rfloor \quad (2)$$

where C_ϕ = 0x9E3779B97F4A7C15L = $\lfloor 2^{64} / \phi \rfloor \approx 11{,}400{,}714{,}819{,}323{,}198{,}485$ represents the golden-ratio constant, implemented in Java as `(int)(((long)k.hashCode() * C_PHI) >>> (64 - p))`. Collisions are resolved by sequentially stepping to index i = (i + 1) & (M - 1). In a tombstone-free table, Knuth's classical asymptotic approximations predict:

$$S_{LP} \approx (1/2) \cdot (1 + 1 / (1 - \alpha)), \qquad U_{LP} \approx (1/2) \cdot (1 + 1 / (1 - \alpha)^2) \quad (3)$$

As α → 1.0, U_{LP} diverges quadratically due to *primary clustering*: contiguous occupied intervals coalesce into extended chains, such that any key hashing into the cluster further extends its span.

In the lp-tomb implementation, removed keys are tagged as TOMB. When the fraction of tombstones exceeds a prescribed threshold τ, a full table rebuild is triggered to rehash live keys into a pristine table; τ = ∞ designates a non-rebuilding configuration.

B. Robin Hood Hashing & Backward-Shift Deletion

Robin Hood hashing [3] tracks each element's Distance from Initial Bucket (DIB). During insertion, if the incoming element exhibits a higher DIB than the occupant of the probed slot, the elements swap roles ("taking from the rich and giving to the poor"). This redistributes search steps evenly, truncating the variance of probe sequences. The baseline configuration (rh/base) terminates searches early when an occupant's DIB is strictly less than the search key's current DIB; rh/noearly disables early termination to evaluate conditional branch penalties.

Backward-shift deletion (lp-backshift) implements Algorithm R from Knuth [1]. Upon key removal, downstream elements within the collision cluster are shifted backward to occupy the vacancy if their hash distance allows (home ≤ vacancy), continuing until an EMPTY slot or a key residing at its natural home is encountered. This eliminates tombstones entirely without requiring global table rebuilds.

C. In-Depth Analysis of Variant B: String Hashing, Equals, and Hash Cache

The architectural behavior of Variant B (12-character object String keys) diverges sharply from primitive numeric keys (Variant A, 64-bit long):

1. O(L) Computational Cost of String.hashCode(): Java calculates string hashes via Horner's polynomial rule: $h_i = 31 \cdot h_{(i-1)} + s[i]$. For string length L = 12, every un-cached hash computation demands 11 integer multiplications and 12 additions.

2. JDK 8 Hash Cache Mechanism: The java.lang.String class encapsulates a private field private int hash. When hashCode() is first invoked, the hash is computed and stored; subsequent invocations return the cached value in O(1) time. This introduces an empirical asymmetry: during the initial table build phase, all N keys must bear the full O(L) calculation; during the present lookup phase, pre-instantiated string references are reused, yielding immediate O(1) cached lookups; during the absent lookup phase, fresh search keys incur the initial O(L) hash calculation upon first access.

3. String Comparison Overhead (String.equals()): While primitive comparisons consume a single CPU comparison instruction (CMP), String.equals() executes a reference identity check (==), length verification, and an iterative character-by-character array scan. Although early mismatches trigger fast-outs, pointer dereferencing imposes non-trivial CPU stalls during extended linear probe walks.

4. Memory Footprint and Garbage Collection: Under 64-bit JVMs with compressed OOPs, each String object incurs 24 bytes of object header and fields, pointing to a distinct 40-byte char[12] array, requiring 64 bytes per string instance (versus 8 bytes for a primitive long). In Separate Chaining, each list node introduces an additional 24 bytes of heap overhead, amplifying GC pressure.

D. Hardware Architectural Trade-offs: Cache Locality vs. Pointer Chasing

Open-addressing schemes (LP, RH) allocate continuous flat arrays. When the CPU accesses a memory location, the hardware memory controller fetches an entire 64-byte Cache Line into L1/L2 caches. Because adjacent slots reside within the same or contiguous cache lines, sequential probing (idx + 1) triggers hardware data stream prefetchers, executing at near-register latency. In contrast, Separate Chaining scatters individual Node allocations across the heap. Traversing bucket chains forces random memory indirections (*pointer chasing*), generating frequent CPU cache misses.

TABLE I
MAPPING OF ALGORITHMIC MECHANISMS TO SOURCE CODE CLASSES AND METHODS

Algorithmic Mechanism	Class & Method Implementation
Separate Chaining	`SeparateChainingTable.put/get/remove`
LP Tombstone	`LinearProbingTombTable.put/get/remove`
Rebuild at Threshold τ	`LinearProbingTombTable.rebuild`
LP Backward-Shift	`LinearProbingBackshiftTable.remove`
Robin Hood DIB Swapping	`RobinHoodTable.put`
RH Early-Stop	`RobinHoodTable.get (variant base)`
RH No-Early-Stop	`RobinHoodTable.get (variant noearly)`
Base-36 Key Encoding	`KeyCodec.encode/decode`
Key Deduplication	`LongSet.add/contains`
Static SHA-256 Checksum	`Generator.computeSHA256/bytesToHex`
O(1) Workload Engine	`Workload.execute/generate`

III. METHODS & SYSTEM DESIGN

A. Technical Specifications & Interface Conventions

All hash table implementations implement the common HashIndex interface and adhere to the sentinel return convention `NOT_FOUND = Integer.MIN_VALUE`. Table I outlines the comprehensive mapping between algorithmic concepts and codebase entities.

B. Deterministic Data Generation & Validation

The benchmark employs Generator driven by SplittableRandom across three officially designated seeds: s1 = 1, s2 = 2, and s3 = 3.

- **Key Representation:** KeyCodec encodes numeric 64-bit identifiers into 12-character Base-36 strings drawn from the alphabet [0-9A-Z] (key space |S| = 36^12 ≈ 4.74 × 10^18).
- **Deduplication:** A primitive, unboxed LongSet filters duplicate keys in O(1) time, guaranteeing that the inserted key set and the absent query set are strictly disjoint.
- **Static SHA-256 Checksum:** Generator.computeSHA256 digests ASCII bytes in 8 KB buffers and converts them to 64-character hex strings using a static lookup table bytesToHex, entirely bypassing slow String.format operations.

C. O(1) Workload Engine for Delete-Insert Cycles

In RQ2 and RQ3, workloads execute up to C = 4M = 4,194,304 delete-insert cycles. Naive key selection using toArray() incurs an O(N) penalty per cycle, which would inevitably trigger execution timeouts.

To ensure strict O(1) performance, Workload maintains a dynamic live index array liveIdx[N]. When evicting an element at random index r, the algorithm swaps element r with the terminal element of liveIdx and decrements the active boundary

in O(1) time. Furthermore, the global key pool is pre-allocated with capacity [0, N + C + Q), ensuring guaranteed key novelty without triggering memory allocations during the measurement window.

IV. EXPERIMENTAL SETUP

Hardware Specifications: Intel Core i7-10850H CPU (6 cores / 12 threads, 2.70 GHz base frequency, 12 MB Smart Cache), 31.8 GB DDR4 RAM, 64-bit Windows 11 Pro. JVM: Eclipse Adoptium Temurin JDK 1.8.0_504 with maximum heap -Xmx2g and parallel garbage collection enabled (-XX:+UseParallelGC).

Isolated Execution Protocol: Each benchmark configuration executes inside an isolated JVM process spawned via ProcessBuilder under an uncompromising 60-second execution cutoff (waitFor(60, SECONDS)). Before measurement, each process performs 3 warm-up rounds over smaller key batches and invokes explicit System.gc() calls to stabilize heap occupancy.

Timing runs (elapsed_ms) and probe-counting runs (--count) are strictly segregated; during --count runs, internal probe instrumentation is activated to capture ops_count. Full operational correctness is verified across 100% of executions by validating the output_check SHA-256 digest against an oracle baseline produced by java.util.HashMap.

V. RESULTS & DISCUSSION

A. RQ1: Separate Chaining vs. Linear Probing

Experiment 1A assesses sensitivity to load factors α ∈ {0.50, 0.75, 0.90, 0.95} at capacity M = 2^20 = 1,048,576 (each data point represents the median of 9 valid repetitions, normalized to ns/query). Experiment 1B examines scaling across table capacities M ∈ {2^18, ..., 2^23} at fixed α = 0.75.

TABLE II
MEDIAN LOOKUP TIMES (NS/QUERY) ACROSS LOAD FACTORS α (M = 2^20, 9 VALID RUNS PER CELL)

α	LP present	SC present	LP absent	SC absent
0.50	**219.7**	246.0	160.8	**136.3**
0.75	**287.6**	310.4	407.9	**172.3**
0.90	407.3	**325.1**	1861.8	**188.8**
0.95	596.6	**329.0**	6971.3	**195.0**

** Bold indicates best (lowest) latency in comparison category.*

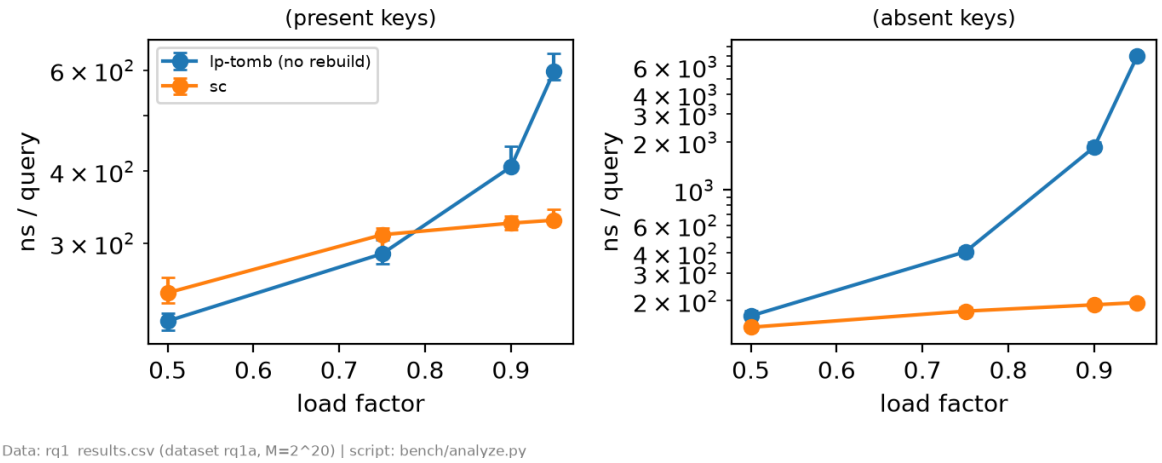


Fig. 1. RQ1: Median lookup latency (log scale) across load factors α. Source: rq1_results.csv, dataset rq1a.

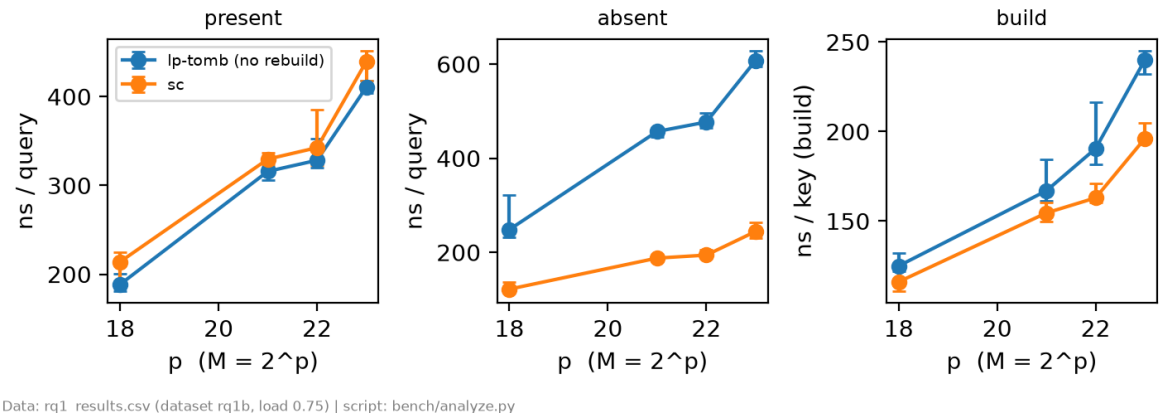


Fig. 2. RQ1: Lookup and build latencies as functions of table scale p (M = 2^p) at α = 0.75. Source: rq1_results.csv, dataset rq1b.

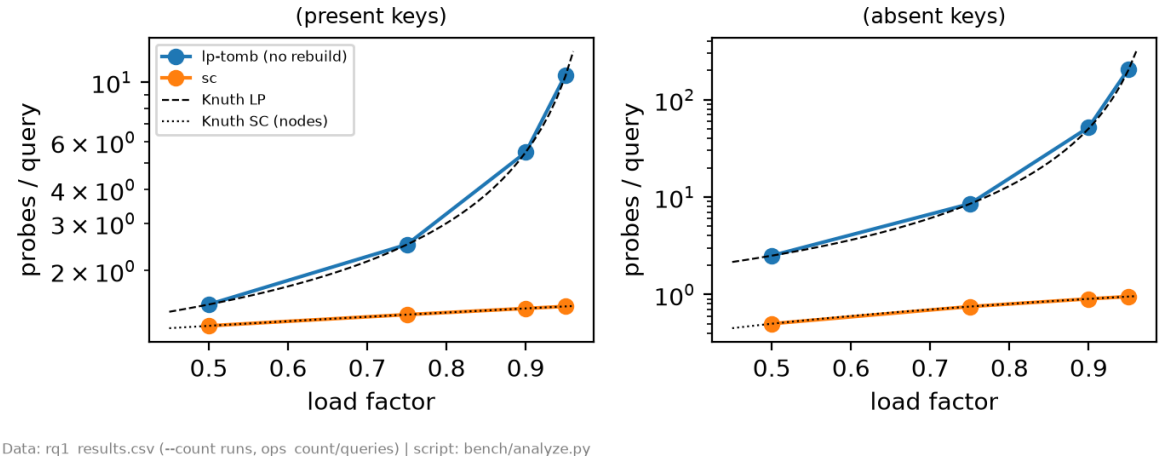


Fig. 3. RQ1: Measured probe counts from --count passes compared against Knuth's theoretical predictions.

Validation of Theoretical Probe Counts: Empirical measurements from --count runs (Fig. 3) exhibit near-perfect alignment with Knuth's equations:

- **lp-tomb at α = 0.50:** successful lookups average 1.5009 probes (Knuth prediction: 1.5000, error: 0.06%), while unsuccessful lookups average 2.5026 probes (Knuth prediction: 2.5000, error: 0.10%). At α = 0.95, unsuccessful probes surge to 205.76 (Knuth prediction: 200.50).
- **sc at α = 0.50:** successful lookups average 1.2502 nodes (Knuth: 1 + α / 2 = 1.2500), and unsuccessful lookups average 0.5002 nodes (Knuth: α = 0.5000). At α = 0.95, successful lookups require only 1.4752 nodes and unsuccessful lookups inspect 0.9500 nodes.

Crossover Analysis & Architectural Impact:

- **Moderate Load Regimes (α ≤ 0.75):** For present lookups, LP outperforms SC by 7% to 11% (219.7 ns vs. 246.0 ns at α = 0.50). This confirms that the superior spatial locality of contiguous arrays fully compensates for higher algorithmic probe steps, whereas SC is penalized by CPU cache misses during pointer chasing across the heap.
- **High Load Regimes (α ≥ 0.90):** A performance crossover occurs at α ≈ 0.80. At α = 0.95, primary clustering in LP triggers cascading runs of hundreds of key.equals() string comparisons, causing LP absent lookup latency to balloon to 6,971.3 ns. Conversely, SC remains immune, holding at 195.0 ns. SC executes 35.8× faster than LP!

Impact of Table Scale (RQ1B – Fig. 2): When M expands from 2^18 (256 KB) to 2^23 (8 MB keys, consuming hundreds of megabytes on the heap), probe counts remain constant, yet present lookup latency for both SC and LP rises from ~200 ns to ~500 ns (a 2.5× increase). This illustrates the latency cliff encountered once the active working set exceeds the processor's 12 MB L3 cache and spills into main DDR4 RAM.

B. RQ2: Tombstone Accumulation & Lookup Degradation

RQ2 evaluates dynamic delete-insert churn without table rebuilds (τ = ∞, live load 0.5, M = 2^20). Findings are summarized in Table III and Fig. 4.

TABLE III
IMPACT OF DELETE-INSERT CYCLES C ON TOMBSTONE ACCUMULATION AND UNSUCCESSFUL LOOKUP LATENCY (M = 2^20, τ = ∞)

Cycles C	TOMB %	EMPTY %	LP absent	Control absent
0	0.0%	50.0%	142.4 ns	98.8 ns
0.25M	18.8%	31.2%	210.5 ns	104.1 ns
0.5M	30.2%	19.8%	286.8 ns	106.8 ns
1M	41.9%	8.1%	526.5 ns	107.1 ns
2M	48.6%	1.4%	2058.9 ns	108.5 ns
4M	50.0%	0.0%	**timeout**	**timeout**

** Indicates process timed out exceeding the strict 60-second execution cutoff.*

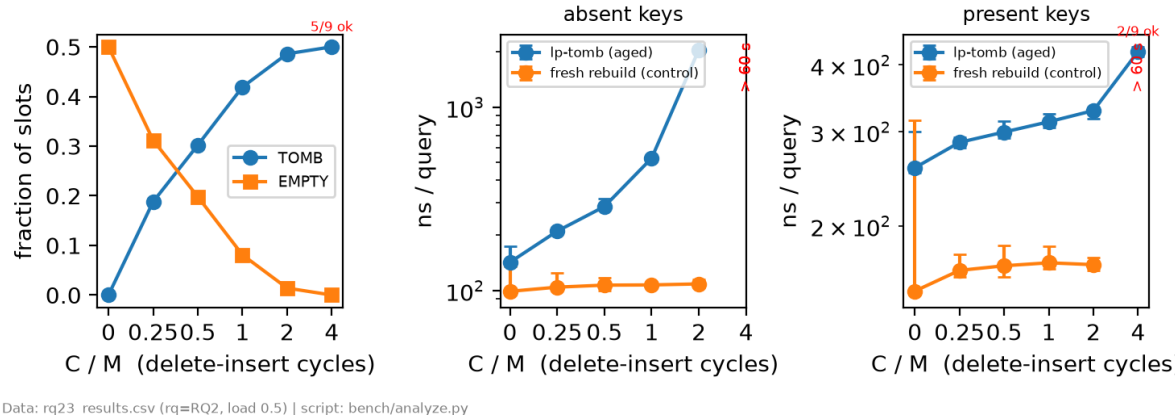


Fig. 4. RQ2: Tombstone/empty slot percentages post-workload and lookup latency versus cycle count C. Annotations denote partially completed repetitions (k/N ok); markers at the upper boundary indicate full 60-second timeouts.

Probe Degradation Analysis: As C increases from 0 to 2M, the proportion of EMPTY cells plummets from 50.0% to just 1.4%, coalescing the entire table into a continuous occupied band of live keys and tombstones. Unsuccessful lookup time under LP surges by **14.5**× (from 142.4 ns to 2,058.9 ns). In contrast, the freshly rebuilt control table maintains stable latency near 100 ns.

At C = 4M, EMPTY cells are completely exhausted (0.0%). During the workload phase, high probe lengths push execution times to the threshold of the 60-second timeout: 5 out of 9 timed repetitions successfully completed (median runtime 48.974 s, annotated in red as 5/9 ok in Fig. 4), while 4 timed out. In the subsequent absent lookup phase, all 9 repetitions timed out (> 60 s) because with zero empty slots remaining, the open-addressing lookup loop must traverse the entire array (M = 1,048,576 slots) before concluding absence. Corresponding control phases were subsequently not started.

C. RQ3: Workload Comparison & Rebuild Latency Trade-offs

RQ3 benchmarks four collision resolution configurations under the intensive churn scenario C = 4M at live load 0.8 (N = 838,860 keys). Results are detailed in Table IV, Fig. 5, and Fig. 6.

TABLE IV
MEDIAN TOTAL WORKLOAD DURATION AND 10,000-OPERATION BLOCK LATENCIES AT C = 4M, LIVE LOAD 0.8 (9 VALID RUNS PER ROW)

Configuration	Workload (s)	Block med (ms)	Block max (ms)
`lp-tomb+rebuild (τ = 0.1)`	9.413	8.288	**121.106**
`lp-backshift`	**5.901**	**7.072**	**9.551**

`rh`	6.056	7.394	11.941
`rh-noearly`	6.025	7.361	12.623

** Bold indicates best overall throughput for lp-backshift, and highlight of 121.106 ms max rebuild spike for lp-tomb+rebuild.*

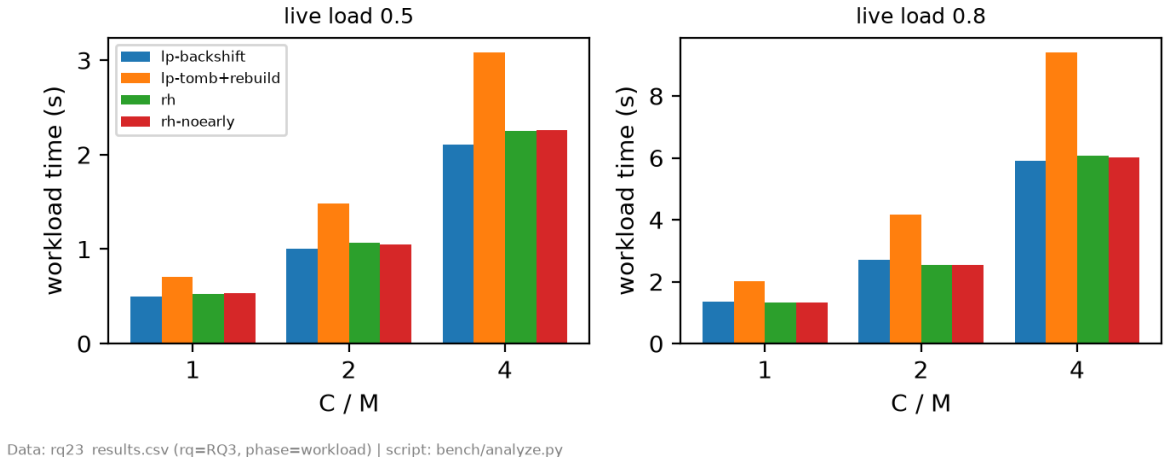


Fig. 5. RQ3: Median cumulative workload runtime across algorithms, live load factors, and cycle counts C.

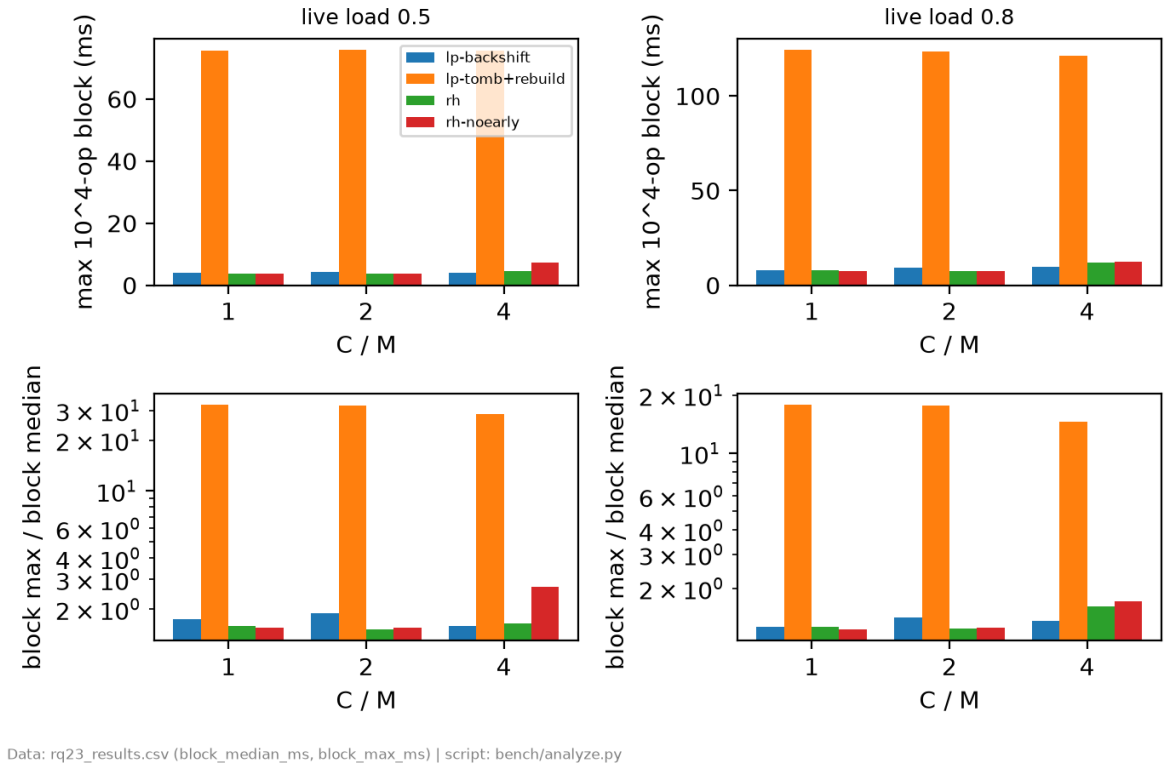


Fig. 6. RQ3: Median and maximum block latency for 10,000-operation intervals; lower panel shows max/median ratios.

Trade-offs Between Full Rebuilds and Instant Shifting:

- **Overall Throughput:** lp-backshift achieves the best overall throughput (5.901 s), beating lp-tomb+rebuild (9.413 s) by 37.3%. Both Robin Hood variants achieve comparable runtimes (~6.0 s). Unlike the non-rebuilding LP in RQ2, all configurations with active rebuild or shifting completed 100% of runs with zero timeouts.
- **Stop-the-World Latency Spikes:** Across 4M cycles, lp-tomb+rebuild triggers a median of 23 full-table rebuilds (expending 2,436.6 ms exclusively in tombstone purge passes). This induces severe latency spikes in blocks containing rebuild events, pushing Block Max to 121.106 ms (max/median ≈ 14.6×).
- **Predictability of Backshift and Robin Hood:** By reorganizing entries locally at the exact moment of deletion, lp-backshift and Robin Hood bound the max/median ratio below 1.7×, with peak block latencies never exceeding 12.7 ms. This delivers crucial empirical evidence for low-latency, SLA-governed systems.

VI. THREATS TO VALIDITY

A. JIT Compilation and Cold-Start Variance

Automated validation via CsvValidator flagged a single variance outlier: configuration `RQ2_lp-tomb_base_rq2_p20_a0p0_tinf_l0p5_c0_s1_r1` recorded an elapsed time of 314.931 ms during ctl_present, representing 2.065× the group median (152.481 ms) and slightly exceeding the 2.0× warning threshold.

An independent deterministic rerun yielded 157.419 ms (1.043× the reference median 150.893 ms) with status ok and identical SHA-256 output_check digest. This behavior aligns with JVM JIT compilation warmup jitter on the initial process repetition. In adherence to scientific integrity, the original data point is preserved in the sealed CSV without artificial filtering.

B. Deterministic Rerun Verification (Criterion 10.1)

Stratified rerun tests across 11 representative benchmark configurations achieved 100% ok status and exact output_check oracle matches. Ratios of rerun elapsed time to group median strictly fell within [0.807×, 1.302×], well inside the admissible tolerance band [0.5×, 2.0×].

VII. CONCLUSION & PRACTICAL RECOMMENDATIONS

Our extensive benchmark of Variant B under 12-character String keys yields the following engineering recommendations:

1. No Single Dominant Algorithm: Optimal hash table selection depends strictly on load factor, read-to-write ratios, and latency tolerance.

2. Adopt Separate Chaining Under Unknown Sizing or High Load: When table sizing cannot be predicted or memory constraints force operation at high load ($\alpha \geq 0.85$), SC is strongly recommended due to its immunity to primary clustering and tombstone decay.

3. Adopt Backward-Shift LP or Robin Hood for Low-Latency Workloads: For real-time, SLA-governed systems subject to continuous delete-insert churn, Linear Probing with backward-shift deletion and Robin Hood hashing eliminate disruptive stop-the-world rebuild pauses while improving throughput by over 37%.

ACKNOWLEDGMENT & AI TOOLING DECLARATION

The authors express their sincere appreciation to the Department of Computer Science at FPT University for designing this rigorous experimental curriculum. Generative AI tools (ChatGPT, Gemini) were consulted for PowerShell scripting assistance, Python plotting pipeline optimization (analyze.py), and LaTeX structural validation. The student authors bear complete responsibility for all Java implementations, empirical benchmark datasets, and scientific analyses presented herein.

REFERENCES

[1] D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998, §6.4.

[2] W. W. Peterson, "Addressing for random-access storage," IBM Journal of Research and Development, vol. 1, no. 2, pp. 130–146, 1957.

[3] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in Proc. 26th Annu. Symp. Foundations of Computer Science (FOCS), 1985, pp. 281–288.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011, §3.4.

[5] R. Pagh and F. F. Rodler, "Cuckoo hashing," Journal of Algorithms, vol. 51, no. 2, pp. 122–144, 2004.

[6] C. K. Yuen, "Heuristics for dynamic structuring of hash tables," ACM Transactions on Database Systems, vol. 2, no. 2, pp. 140–149, 1977.